Lecture 3

C# Strings

Strings are used for storing text.

A `string` variable contains a collection of characters surrounded by double quotes:

Code:

```
using System;

namespace MyApplication
{
  class Program
  {
    static void Main(string[] args)
    {
      string greeting = "Hello";
      Console.WriteLine(greeting);
    }
  }
}
```

Output:



- ### String Length

A string in C# is actually an object, which contain properties and methods that can perform certain operations on strings. For example, the length of a string can be found with the Length property:

Code:

```
string txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

Console.WriteLine("The length of the txt string is: " + txt.Length);
```

Output:

```
The length of the txt string is: 26
```

- ## Other Methods

There are many string methods available, for example ToUpper() and ToLower(), which returns a copy of the string converted to uppercase or lowercase:

Code

```
string txt = "Hello World";

Console.WriteLine(txt.ToUpper());   // Outputs "HELLO WORLD"

Console.WriteLine(txt.ToLower());   // Outputs "hello world"
```

- ## String Concatenation

The + operator can be used between strings to combine them. This is called **concatenation**:

Code:

```
string firstName = "John ";

string lastName = "Doe";

string name = firstName + lastName;

Console.WriteLine(name);
```

You can also use the string.Concat() method to concatenate two strings:

```
string firstName = "John ";

string lastName = "Doe";

string name = string.Concat(firstName, lastName);

Console.WriteLine(name);
```

- ## String Interpolation

Another option of string concatenation, is **string interpolation**, which substitutes values of variables into placeholders in a string. Note that you do not have to worry about spaces, like with concatenation:

Code:

```
string firstName = "John";

string lastName = "Doe";

string name = $"My full name is: {firstName} {lastName}";

Console.WriteLine(name);
```

- ## Access Strings

Example:

```
string myString = "Hello";

Console.WriteLine(myString[0]);  // Outputs "H"
```

Example

```
string myString = "Hello";

Console.WriteLine(myString.IndexOf("e"));  // Outputs "1"
```

Another useful method is Substring(), which extracts the characters from a string, starting from the specified character position/index, and returns a new string. This method is often used together with IndexOf() to get the specific character position:

Example

```
// Full name

string name = "John Doe";



// Location of the letter D

int charPos = name.IndexOf("D");
```

```
// Get last name
string lastName = name.Substring(charPos);

// Print the result
Console.WriteLine(lastName);
```

- **Special Characters**

Example:

```
string txt = "We are the so-called "Vikings" from the north.";
string txt = "We are the so-called \"Vikings\" from the north.";
string txt = "It\'s alright.";
```

Output?

C# If ... Else

```
int time = 20;
if (time < 18)
{
  Console.WriteLine("Good day.");
}
else
{
  Console.WriteLine("Good evening.");
}
// Outputs "Good evening."
```

- **Nested if else:**

```
int time = 22;
if (time < 10)
{
  Console.WriteLine("Good morning.");
}
else if (time < 20)
{
  Console.WriteLine("Good day.");
}
else
{
  Console.WriteLine("Good evening.");
}
// Outputs "Good evening."
```

- ## Short Hand If...Else (Ternary Operator)

There is also a short-hand if else, which is known as the **ternary operator** because it consists of three operands. It can be used to replace multiple lines of code with a single line. It is often used to replace simple if else statements:

```
int time = 20;
string result = (time < 18) ? "Good day." : "Good evening.";
Console.WriteLine(result);
```

C# Switch

Use the switch statement to select one of many code blocks to be executed.

Example

```
int day = 4;
switch (day)
```

```
{
  case 1:
    Console.WriteLine("Monday");
    break;
  case 2:
    Console.WriteLine("Tuesday");
    break;
  case 3:
    Console.WriteLine("Wednesday");
    break;
  case 4:
    Console.WriteLine("Thursday");
    break;
  case 5:
    Console.WriteLine("Friday");
    break;
  case 6:
    Console.WriteLine("Saturday");
    break;
  case 7:
    Console.WriteLine("Sunday");
    break;
}
// Outputs "Thursday" (day 4)
```

- ## The break Keyword

When C# reaches a `break` keyword, it breaks out of the switch block.

This will stop the execution of more code and case testing inside the block.

When a match is found, and the job is done, it's time for a break. There is no need for more testing.

A break can save a lot of execution time because it "ignores" the execution of all the rest of the code in the switch block.

- ## The default Keyword

The `default` keyword is optional and specifies some code to run if there is no case match.

C# Loops

- ## C# While Loop

Example

```
int i = 0;

while (i < 5)

{

 Console.WriteLine(i);

 i++;

}
```

- ## The Do/While Loop

```
int i = 0;
do

{

 Console.WriteLine(i);

 i++;

}
```

```
while (i < 5);
```

Do not forget to increase the variable used in the condition, otherwise the loop will never end!

- ## C# For Loop

Example

```
for (int i = 0; i < 5; i++)
{
  Console.WriteLine(i);
}
```

- ## The foreach Loop

There is also a foreach loop, which is used exclusively to loop through elements in an **array.**

Example

```
string[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
foreach (string i in cars)
{
  Console.WriteLine(i);
}
```

C# Break and Continue

- ## C# Break

The break statement can also be used to jump out of a **loop, switch.**

Example

```
for (int i = 0; i < 10; i++)
{
  if (i == 4)
  {
```

```
  break;
 }
 Console.WriteLine(i);
}
```

Output?

- ## C# Continue

The `continue` statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Example

```
for (int i = 0; i < 10; i++)
{
 if (i == 4)
 {
  continue;
 }
 Console.WriteLine(i);
}
```

Output?

Lab Task:

1. Create a basic calculator in C# Console. Input must be taken from user for every calculation.
2. Ask user to enter 2 or 3 values in your calculator and using comparison operators return true or false by comparing user values with one another.

3. Use Math functions to perform advance functions of calculator i.e. rounding off a value, calculating square root finding out max or min value from multiple use inputs. To store values for constant variables use constants of C# i.e. pi value etc.
4. Print outputs for all the tasks individually.

Note: last half hour is dedicated for viva.